

Week 2

Selection Components: JCheckBox, JRadioButton, JComboBox, JList

What's on today

Nine components this week — every one, on its own and combined, with the OOP idea it demonstrates explained right alongside it.

01–04

Selection

JCheckBox, JRadioButton, JComboBox, JList

05

Text area

JTextArea + JScrollPane

06–08

Value components

JSlider, JSpinner, JProgressBar

09

Four interfaces

Action / Item / ListSelection / Change

10

Two combined examples

Game Settings panel + Game HUD

11

Lab & mistakes

Common pitfalls, this week's hands-on targets

Quick Recap: Where We Left Off

Components

JLabel, JButton, JTextField,
JPasswordField — plus JFrame and
JPanel to hold them.

Events

Click → `ActionEvent` →
`ActionListener.actionPerformed()` →
GUI updates.

OOP pillars

Inheritance (the Component tree),
Polymorphism (`add(Component)`),
Interface (`ActionListener`),
Encapsulation (private fields +
getters/setters).

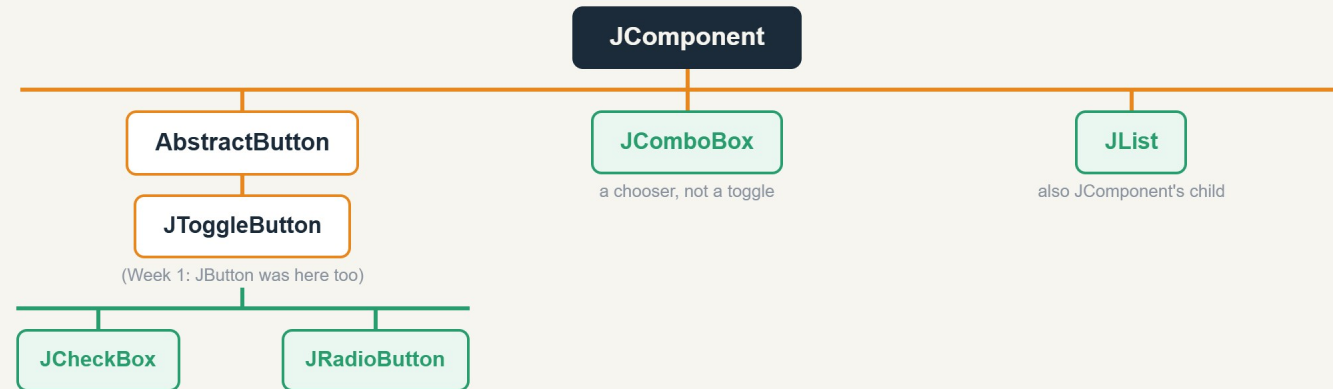
Layouts

`FlowLayout`, `BorderLayout`,
`GridLayout` — and null layout for later.

Today's components sit on the exact same class tree and fire events through the exact same kind of interface — just with new shapes and new contracts.

Where Do Today's Components Fit in the Tree?

Remember Week 1's Swing class tree? JCheckBox and JRadioButton graft onto the same AbstractButton branch as JButton — JComboBox and JList grow from JComponent directly, a separate branch.



The branch a class sits on tells you what it inherits for free: JCheckBox gets `isSelected()/setSelected()` from AbstractButton, the exact pair JButton's ancestor already had.

Component Deep Dive: JCheckBox

Scenario: the player wants sound on or off — an option that has nothing to do with any other option.

```
import javax.swing.*;

public class CheckBoxDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Sound Settings");
        JPanel panel = new JPanel();

        JCheckBox soundBox = new JCheckBox("Sound on", true);
        JLabel status = new JLabel("Sound: ON");

        soundBox.addItemListener(e -> {
            boolean on = soundBox.isSelected();
            status.setText(on ? "Sound: ON" : "Sound: OFF");
        });

        panel.add(soundBox); panel.add(status);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

☒ Sound on **Sound: ON**

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JCheckBox extends JToggleButton extends AbstractButton, the same branch JButton sits on.
- 2. Interface** — it fires an ItemEvent through ItemListener, a different contract than the ActionListener we used for JButton.
- 3. Encapsulation** — its checked state is a private field, reachable only via isSelected() / setSelected(boolean).

Why Isn't ButtonGroup a Component?

"I tried `panel.add(group)` and it wouldn't compile — why not?"

A **Component** is something that gets drawn on screen and lives in the container tree (JFrame → JPanel → ...).

A **ButtonGroup** draws nothing. It's a plain Java object that just keeps a list of buttons and deselects the others whenever one is picked.

This is **composition**, not inheritance: ButtonGroup doesn't extend Component and add new drawing behavior — it just holds references to components and calls their existing public methods (`setSelected(false)`) from outside.

TWO SEPARATE THINGS ON SCREEN

```
panel (a Container — visible)
```

```
├─ easy      (JRadioButton)
```

```
├─ medium   (JRadioButton)
```

```
└─ hard     (JRadioButton)
```

```
group (a ButtonGroup — invisible)
```

```
  knows about: easy, medium, hard
```

Component Deep Dive: JRadioButton + ButtonGroup

Scenario: the player picks a difficulty — Easy, Medium or Hard, but only ever one at a time.

```
import javax.swing.*;

public class RadioButtonDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Difficulty");
        JPanel panel = new JPanel();

        JRadioButton easy = new JRadioButton("Easy");
        JRadioButton medium = new JRadioButton("Medium", true);
        JRadioButton hard = new JRadioButton("Hard");

        ButtonGroup group = new ButtonGroup();
        group.add(easy); group.add(medium); group.add(hard);

        JLabel status = new JLabel("Difficulty: Medium");
        easy.addActionListener(e -> status.setText("Difficulty: Easy"));
        medium.addActionListener(e -> status.setText("Difficulty: Medium"));
        hard.addActionListener(e -> status.setText("Difficulty: Hard"));

        panel.add(easy); panel.add(medium); panel.add(hard); panel.add(status);
        frame.add(panel);
        frame.setSize(380, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

☐ Easy ☒ Medium ☐ Hard

Difficulty: Medium

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JRadioButton also extends JToggleButton — a sibling of JCheckBox on the same branch.
- 2. Composition, not inheritance** — ButtonGroup isn't a Component at all; it's a plain helper object that just deselects its other members.
- 3. Encapsulation** — ButtonGroup enforces "only one selected" from outside, using each button's own public isSelected()/setSelected().

Component Deep Dive: JComboBox

Scenario: the player picks their character's color from a short dropdown list.

```
import javax.swing.*;
import java.awt.event.ItemEvent;

public class ComboBoxDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Character Color");
        JPanel panel = new JPanel();

        String[] colors = {"Red", "Blue", "Green", "Yellow"};
        JComboBox<String> colorBox = new JComboBox<>(colors);
        JLabel status = new JLabel("Color: Red");

        colorBox.addItemListener(e -> {
            if (e.getStateChange() == ItemEvent.SELECTED) {
                status.setText("Color: " + colorBox.getSelectedItem());
            }
        });

        panel.add(colorBox); panel.add(status);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Blue ▾ Color: Blue

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JComboBox extends JComponent directly — a different branch than the buttons; it's a chooser, not a toggle.
- 2. Polymorphism** — it fires the same ItemListener contract JCheckBox used — one interface, reused across very different-looking components.
- 3. Encapsulation** — the chosen item is reachable only via `getSelectedItem()`, never a public field.

Component Deep Dive: JList

Scenario: the player picks their profile from a scoreboard of existing players.

```
import javax.swing.*;

public class ListDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Scoreboard");
        JPanel panel = new JPanel();

        String[] players = {"Ayse: 120", "Mehmet: 95", "Zeynep: 140"};
        JList<String> playerList = new JList<>(players);
        JLabel status = new JLabel("Pick a player");

        playerList.addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) {
                status.setText("Selected: " + playerList.getSelectedValue());
            }
        });

        panel.add(playerList); panel.add(status);
        frame.add(panel);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Ayse: 120

Mehmet: 95

Zeynep: 140

Selected: Mehmet: 95

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JList also extends JComponent directly, the same branch as JComboBox — both are choosers.
- 2. A third interface** — ListSelectionListener has its own method, valueChanged() — same idea as ActionListener/ItemListener, new contract.
- 3. Encapsulation** — getSelectedValue() is the only door to the current pick.

Putting It All Together: a Game Settings Panel

```
import javax.swing.*;

public class GameSettings {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Game Settings");
        JPanel panel = new JPanel();

        JCheckBox soundBox = new JCheckBox("Sound", true);

        JRadioButton easy = new JRadioButton("Easy");
        JRadioButton medium = new JRadioButton("Medium", true);
        JRadioButton hard = new JRadioButton("Hard");
        ButtonGroup diff = new ButtonGroup();
        diff.add(easy); diff.add(medium); diff.add(hard);

        String[] colors = {"Red", "Blue", "Green"};
        JComboBox<String> colorBox = new JComboBox<>(colors);

        JButton startBtn = new JButton("Start");
        JLabel summary = new JLabel(" ");

        startBtn.addActionListener(e -> {
            String diffText = easy.isSelected() ? "Easy" :
                hard.isSelected() ? "Hard" : "Medium";
            summary.setText("Sound=" + soundBox.isSelected()
                + ", Diff=" + diffText
                + ", Color=" + colorBox.getSelectedItem());
        });

        panel.add(soundBox);
        panel.add(easy); panel.add(medium); panel.add(hard);
        panel.add(colorBox);
        panel.add(startBtn); panel.add(summary);

        frame.add(panel);
        frame.setSize(420, 220);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT



☒ Sound

☐ Easy ☒ Medium ☐ Hard

Blue ▾

Start

Sound=true, Diff=Medium, Color=Blue

One ActionListener on the Start button reads **four** different components — each through its own encapsulated getter (`isSelected()`, `getSelectedItem()`) — and combines them into one summary string.

This is the same "one handler, many inputs" shape as Week 1's login form — now with selection components instead of text fields.

Why Do We Need JScrollPane?

"My JTextArea has more lines than fit — why don't I just make the window bigger?"

A **JTextArea** (and **JList**) can hold far more content than fits on screen — you can't resize the window every time.

JScrollPane wraps another component and adds scrollbars automatically when the content overflows its own fixed size.

You never add a `JTextArea` straight to a `JPanel` when it might overflow — you add it to a `JScrollPane`, and add *that* to the panel instead.

ONE MORE LAYER IN THE HIERARCHY

`JFrame`

└─ `JPanel`

└─ `JScrollPane`

└─ `JTextArea` (or `JList`)

```
new JScrollPane(textArea)
```

Component Deep Dive: JTextArea

Scenario: a running game log — "Player joined", "Egg caught!", "Game over" — appended line by line.

```
import javax.swing.*;

public class TextAreaDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Game Log");
        JPanel panel = new JPanel();

        JTextArea log = new JTextArea(6, 22);
        log.setEditable(false);
        log.setLineWrap(true);

        JButton addLine = new JButton("Egg caught!");
        addLine.addActionListener(e ->
            log.append("Egg caught!\n"));

        panel.add(new JScrollPane(log));
        panel.add(addLine);
        frame.add(panel);
        frame.setSize(320, 260);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Player joined
Egg caught!
Egg caught!

Egg caught!

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JTextArea extends JTextComponent extends JComponent — the same branch JTextField was on in Week 1.
- 2. Inherited behavior** — that's why getText()/setText() work exactly as they did for JTextField; append() is JTextArea's own addition.
- 3. Composition** — new JScrollPane(log) wraps it — the scroll pane isn't a subclass of JTextArea, it just holds a reference to one.

JTextField vs. JTextArea

Both are JTextComponent's children — same tree, same getText()/setText() — but built for different jobs.

Aspect	JTextField (Week 1)	JTextArea (today)
Lines	One line only	Many lines, wraps and scrolls
Typical use	Username, a single value	A log, a comment box, a description
Add content	setText(String) replaces it	append(String) adds without replacing
Needs JScrollPane?	No — it's already one line	Almost always, once content can overflow

Shared ancestor, different personality — a good example of how one branch of the tree can still hold very different-feeling components.

Component Deep Dive: JScrollPane

Scenario: a scoreboard with far more players than fit in the visible window.

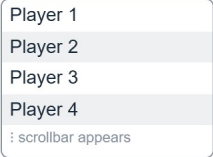
```
import javax.swing.*;

public class JScrollPaneDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Full Scoreboard");
        JPanel panel = new JPanel();

        String[] players = new String[20];
        for (int i = 0; i < 20; i++)
            players[i] = "Player " + (i + 1);

        JList<String> list = new JList<>(players);
        JScrollPane scrollPane = new JScrollPane(list);
        scrollPane.setPreferredSize(new Dimension(200, 150));

        panel.add(scrollPane);
        frame.add(panel);
        frame.setSize(280, 250);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



Player 1
Player 2
Player 3
Player 4
: scrollbar appears

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Composition, not inheritance** — JScrollPane doesn't extend JList; its constructor takes any Component and holds a reference to it.
- 2. Polymorphism** — that constructor's parameter type is Component — it happily wraps a JList today, a JTextArea a slide ago, or any future component.
- 3. It's the wrapper you add** — panel.add(scrollPane), never panel.add(list) directly, once scrolling matters.

Component Deep Dive: JSlider

Scenario: the player drags a slider to set the game's volume from 0 to 100.

```
import javax.swing.*;

public class SliderDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Volume");
        JPanel panel = new JPanel();

        JSlider volume = new JSlider(0, 100, 70);
        volume.setMajorTickSpacing(25);
        volume.setPaintTicks(true);

        JLabel status = new JLabel("Volume: 70");
        volume.addChangeListener(e ->
            status.setText("Volume: " + volume.getValue()));

        panel.add(volume); panel.add(status);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

0 25 50 75 100

Volume: 70

STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JSlider extends JComponent directly — its own branch, built to represent a numeric range, not a toggle or a chooser.
- 2. A fourth interface** — ChangeListener's one method, stateChanged(), fires continuously while dragging — a new contract for "value" components.
- 3. Encapsulation** — the current position is a private int, reachable only via getValue()/setValue(int).

Component Deep Dive: JSpinner

Scenario: the player picks how many lives to start with — a small, precise number, not a rough position on a bar.

```
import javax.swing.*;

public class SpinnerDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Starting Lives");
        JPanel panel = new JPanel();

        SpinnerNumberModel model =
            new SpinnerNumberModel(3, 1, 9, 1);
        JSpinner livesSpinner = new JSpinner(model);

        JLabel status = new JLabel("Lives: 3");
        livesSpinner.addChangeListener(e -> {
            int lives = (Integer) livesSpinner.getValue();
            status.setText("Lives: " + lives);
        });

        panel.add(livesSpinner); panel.add(status);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JSpinner extends JComponent directly, like JSlider — but is built for precise, discrete values.
- 2. Composition** — a JSpinner doesn't hold its own min/max/step; it delegates to a separate SpinnerNumberModel object it's given.
- 3. Polymorphism** — getValue() returns a plain Object — you cast it to Integer because you know what model you gave it.

Component Deep Dive: JProgressBar

Scenario: a health bar that fills or drains as a "Take Damage" button is clicked — the player never types or drags it.

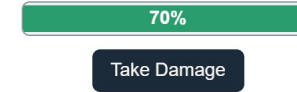
```
import javax.swing.*;

public class ProgressBarDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Health");
        JPanel panel = new JPanel();

        JProgressBar health = new JProgressBar(0, 100);
        health.setValue(100);
        health.setStringPainted(true);

        JButton hit = new JButton("Take Damage");
        hit.addActionListener(e -> {
            int newValue = health.getValue() - 15;
            health.setValue(Math.max(0, newValue));
        });

        panel.add(health); panel.add(hit);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



STEP BY STEP — WHERE OOP SHOWS UP

- 1. Inheritance** — JProgressBar extends JComponent directly too — same branch as JSlider, but purely for display.
- 2. No listener needed** — nothing selects a JProgressBar, so it never fires ChangeListener the way JSlider does — a good contrast: not every component listens for input.
- 3. Encapsulation** — its fill level is set only through setValue(int), same shape as every other component we've met.

Four Interfaces, One Idea

Week 1: "implement one method, Swing calls you back." Every component we've met fires through one of these four contracts.

Interface	Method	Fires when...	Used with
ActionListener	actionPerformed(ActionEvent e)	the component is "activated" (click, Enter)	JButton, JTextField, JRadioButton
ItemListener	itemStateChanged(ItemEvent e)	the selection state changes	JCheckBox, JComboBox
ListSelectionListener	valueChanged(ListSelectionEvent e)	the highlighted row(s) change	JList
ChangeListener	stateChanged(ChangeEvent e)	a numeric value changes, often continuously	JSlider, JSpinner

JCheckBox and JRadioButton actually support both ActionListener and ItemListener. JProgressBar fires none of these — nothing selects it, so it never needs a listener at all.

Putting It All Together: a Game HUD

```
import javax.swing.*;

public class GameHud {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Game HUD");
        JPanel panel = new JPanel();

        JSlider volume = new JSlider(0, 100, 70);
        JSpinner lives = new JSpinner(
            new SpinnerNumberModel(3, 1, 9, 1));
        JProgressBar health = new JProgressBar(0, 100);
        health.setValue(100);
        health.setStringPainted(true);
        JTextArea log = new JTextArea(4, 20);
        log.setEditable(false);

        JButton hit = new JButton("Take Damage");
        hit.addActionListener(e -> {
            health.setValue(Math.max(0, health.getValue() - 15));
            log.append("Took damage! HP=" + health.getValue() + "\n");
        });

        panel.add(new JLabel("Vol")); panel.add(volume);
        panel.add(new JLabel("Lives")); panel.add(lives);
        panel.add(health); panel.add(hit);
        panel.add(new JScrollPane(log));

        frame.add(panel);
        frame.setSize(360, 320);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

OUTPUT

Vol: —●— 70

Lives: [3 ▲▼]

55%

Take Damage

Took damage! HP=70
Took damage! HP=55

Four components, four different jobs — one ActionListener reads the button click, updates the JProgressBar's value, and appends a line to the JTextArea log, all in one method body.

Same shape as the login form and the settings panel: **read state** → **compute** → **write state**, just with new components each time.

Reference: Nine Components at a Glance

Component	Extends	Listener	Key method
JCheckBox	JToggleButton	ItemListener	isSelected()
JRadioButton	JToggleButton	ActionListener	isSelected()
JComboBox	JComponent	ItemListener	getSelectedItem()
JList	JComponent	ListSelectionListener	getSelectedValue()
JTextArea	JTextComponent	(usually none)	append(String)
JScrollPane	JComponent	—	new JScrollPane(comp)
JSlider	JComponent	ChangeListener	getValue()
JSpinner	JComponent	ChangeListener	getValue()
JProgressBar	JComponent	—	setValue(int)

Two Weeks of Components, Four Families

Thirteen components so far — every single one still traces back to Component. Grouped by branch, the tree gets easier to hold in your head.

Toggle family

AbstractButton → JToggleButton

JButton (Wk1)
JCheckBox
JRadioButton

Text family

JComponent → JTextComponent

JTextField (Wk1)
JPasswordField (Wk1)
JTextArea

Chooser family

JComponent, directly

JComboBox
JList

Value family

JComponent, directly

JSlider
JSpinner
JProgressBar

JLabel and JScrollPane don't fit any family above — JLabel is passive (no listener at all), and JScrollPane is a wrapper that holds another component rather than being one of these "input" types.

Building a Settings Screen in Layers

With nine components on one screen, a single flat JPanel gets messy fast. Nesting panels — Week 1's JFrame/JPanel idea, one level deeper — keeps it organized.

```
JFrame
├── mainPanel (JPanel)
│   ├── audioPanel (JPanel)
│   │   ├── JCheckBox (sound)
│   │   └── JSlider (volume)
│   ├── difficultyPanel (JPanel)
│   │   └── JRadioButton × 3 + ButtonGroup
│   └── statusPanel (JPanel)
│       └── JScrollPane → JTextArea (log)
```

Each inner JPanel gets its **own layout** — audioPanel might use FlowLayout while mainPanel uses BorderLayout to stack the three sections vertically.

This is **composition** again — panels containing panels containing components — the same containment idea from Week 1, just nested one level deeper.

The Game Settings and Game HUD examples we just saw used one flat panel to fit on a slide — in a real project, group them like this instead.

The 5 Steps Still Apply

Week 1's recipe didn't change — only step 3 got a lot more interesting.

- 1 Create the window — `JFrame frame = new JFrame();`
- 2 Create a container — `JPanel panel = new JPanel();`
- 3 **Create the components — now nine kinds to pick from, not four**
- 4 Add components → panel → frame
- 5 Show the window — `frame.setVisible(true);`

Whatever component you reach for — `JCheckBox` or `JProgressBar` — it still goes through these same five steps. That consistency is exactly what the shared `Component` ancestor buys you.

Common Mistakes I — Selection Components

Forgot to add the radio buttons to a ButtonGroup

Symptom: all three difficulty buttons can be checked at once — they behave like checkboxes, not radio buttons.

Fix: create a ButtonGroup and call group.add(...) for every JRadioButton in the set — the group is a separate object, not automatic.

Used addActionListener() on a JCheckBox and expected the new state

Symptom: the listener runs, but there's no direct way to read "was it just checked or unchecked?" from an ActionEvent.

Fix: use addItemListener() when you need the state itself, and read it with isSelected() — or call isSelected() manually inside the ActionListener.

Forgot to check getValuesAdjusting() on a JList

Symptom: the handler fires several times for what feels like one click-and-drag selection.

Fix: wrap the handler body in if (!e.getValuesAdjusting()) so it runs once the selection has settled.

Common Mistakes II — Text & Value Components

Added a JTextArea or JList straight to the panel

Symptom: long content is silently clipped — no scrollbar appears, and there's no way to reach the rest.

Fix: wrap it first — `panel.add(new JScrollPane(myTextArea))`, never `panel.add(myTextArea)` directly once it can overflow.

Forgot to cast JSpinner's getValue()

Symptom: `int lives = spinner.getValue();` doesn't compile — `getValue()` returns a plain `Object`.

Fix: cast explicitly — `int lives = (Integer) spinner.getValue();`

JProgressBar shows an empty bar with no text

Symptom: the bar fills correctly, but no percentage or label is visible on it.

Fix: call `setStringPainted(true)` — the text overlay is opt-in, not automatic.

Common Mistakes III — Sliders & Scroll Panes

Set tick spacing but no ticks appear on the JSlider

Symptom: `setMajorTickSpacing(25)` is called, but the slider still looks like a bare bar.

Fix: spacing alone isn't enough — also call `setPaintTicks(true)` (and `setPaintLabels(true)` for numbers).

A ChangeListener fires dozens of times per drag

Symptom: expensive work in the handler (like updating a log) runs constantly while the thumb is being dragged, not just when released.

Fix: guard with `if (!slider.getValueIsAdjusting())` — JSlider has this method too, same idea as JList's version from earlier.

A JScrollPane collapses to almost nothing

Symptom: the scroll pane appears as a tiny sliver instead of a reasonably sized box.

Fix: give it an explicit size — `scrollPane.setPreferredSize(new Dimension(w, h))` — it won't guess one for you.

Lab Goals I — Selection Components

- Build a JCheckBox demo and toggle its state with ItemListener
- Build a JRadioButton + ButtonGroup demo — then remove the group and see it break
- Build a JComboBox demo reading the selected item with ItemListener
- Build a JList demo, guarding the handler with getValuesAdjusting()
- Combine all four into the Game Settings panel with a working Start button

Lab Goals — Text & Scrolling

- Build a JTextArea demo that appends a new line on each button click
- Wrap a 20-item JList in a JScrollPane and confirm the scrollbar appears
- Compare: add the same JTextArea with and without a JScrollPane wrapper
- Note down: which components today needed a JScrollPane and which didn't, and why

Lab Goals — Value Components & Wrap-Up

- Build a JSlider demo with tick marks, guarded by `getValueIsAdjusting()`
- Build a JSpinner demo with a `SpinnerNumberModel` and a correct cast
- Build a JProgressBar demo that drains on a button click, with `setStringPainted(true)`
- Combine into the Game HUD: volume + lives + health + log, in nested panels
- For each of the nine components, name which listener interface it uses (or none)

NEXT WEEK

Week 3: Timer & Animation Basics

With all nine components covered, we move to `javax.swing.Timer` and the game loop — the first moving shape on screen, one step closer to the egg-catching game.

Questions?